

Modül 15

Animations

Modern Angular Animation API

Modül: 15 / 30

Ders Sayısı: 6 ders

Seviye: Orta

Versiyon: Angular v21

Hazırlayan: Claude • Türkçe Angular v21 Eğitimi

Modül 15'e Hoş Geldin

Modern Angular'da animasyonlar için iki yaklaşım: Angular'ın kendi @angular/animations paketi (klasik) ve CSS-based + View Transitions API (modern, önerilen). Bu modülde her ikisini de işliyoruz ama modern yaklaşıma odaklanıyoruz.

Öğrenecekler:

- ✓ CSS animations + transitions (en yaygın, en performant)
- ✓ @if, @for blokları içinde animation
- ✓ Route animations (View Transitions API)
- ✓ Angular Animations API (klasik trigger sistem)
- ✓ Reduce motion accessibility
- ✓ Performance tips (transform, opacity tercih)

CSS Animations & Transitions

En basit ve performant yaklaşım: pure CSS.

□ CSS Transition

```
.button {
  background: #4f46e5;
  transition: all 200ms ease;
}

.button:hover {
  background: #6366f1;
  transform: scale(1.05);
}

/* Modern: aspect-ratio, color-mix, vs */
.card {
  transition: transform 300ms cubic-bezier(0.4, 0, 0.2, 1);
}
.card:hover {
  transform: translateY(-4px);
}
```

□ CSS Keyframe Animation

```
@keyframes fadeIn {
  from { opacity: 0; transform: translateY(10px); }
  to { opacity: 1; transform: translateY(0); }
}

.fade-in {
  animation: fadeIn 300ms ease;
}

/* Stagger animation */
.list-item {
  animation: fadeIn 400ms ease backwards;
}
.list-item:nth-child(1) { animation-delay: 0ms; }
.list-item:nth-child(2) { animation-delay: 50ms; }
.list-item:nth-child(3) { animation-delay: 100ms; }
```

⚡ Performance — GPU Acceleration

✓ transform ve opacity — GPU accelerated

- ✓ top/left/margin — CPU, layout trigger
- ✓ will-change: transform — browser hint
- ✓ transform: translate3d(0,0,0) — eski hile (artık gerek yok)

□ **Anti-Patterns**

- ✗ left/top değiştirme animasyon için (layout thrashing)
- ✗ height/width animasyon (yerine transform: scale)
- ✗ box-shadow animasyon (yerine ::before/::after)
- ✗ 60fps üzerinde animation (gereksiz)

@if / @for ile Animation

Modern Angular'da enter/leave animasyonları.

□ Enter/Leave Class

```
<!-- Component template -->
@if (showModal()) {
  <div class="modal modal-enter">
    Modal içerik
  </div>
}

<!-- CSS -->
<style>
.modal {
  opacity: 0;
  transform: scale(0.9);
}

.modal-enter {
  animation: modal-in 200ms ease forwards;
}

@keyframes modal-in {
  to {
    opacity: 1;
    transform: scale(1);
  }
}
</style>
```

□ List Stagger

```
<ul>
  @for (item of items(); track item.id; let i = $index) {
    <li
      class="item"
      [style.animation-delay.ms]="i * 50">
      {{ item.name }}
    </li>
  }
</ul>

<style>
.item {
  animation: slide-in 300ms ease backwards;
}
@keyframes slide-in {
  from { opacity: 0; transform: translateX(-20px); }
  to { opacity: 1; transform: translateX(0); }
}
</style>
```

Route Animations

View Transitions API — modern, native browser desteği.

□ View Transitions Setup

```
import { provideRouter, withViewTransitions } from '@angular/router';

provideRouter(routes,
  withViewTransitions({
    skipInitialTransition: true,
    onViewTransitionCreated: ({ transition, from, to }) => {
      console.log(`Navigating from ${from.url} to ${to.url}`);
    }
  })
)
```

□ CSS — Transition Styles

```
/* Root level transition */
::view-transition-old(root) {
  animation: fade-out 200ms ease forwards;
}

::view-transition-new(root) {
  animation: fade-in 200ms ease forwards;
}

@keyframes fade-out {
  to { opacity: 0; }
}
@keyframes fade-in {
  from { opacity: 0; }
  to { opacity: 1; }
}

/* Named transition (shared element) */
.hero-image {
  view-transition-name: hero;
}

::view-transition-old(hero),
::view-transition-new(hero) {
  animation-duration: 400ms;
}
```

⚠ **Browser Support**

📦 **Browser Compatibility**

View Transitions API Chrome 111+ ve Edge'de destekli. Safari ve Firefox'ta yok. Angular fallback yapar — animasyon olmadan navigation.

Angular Animations API (Klasik)

Eski ama hâlâ kullanılan @angular/animations paketi.

⚙ Setup

```
// app.config.ts
import { provideAnimationsAsync } from '@angular/platform-browser/animations/async';

providers: [
  provideAnimationsAsync(), // Modern, lazy load
]
```

□ Trigger Tanımla

```
import { trigger, state, style, transition, animate } from '@angular/animations';

@Component({
  animations: [
    trigger('openClose', [
      state('open', style({ height: '200px', opacity: 1 })),
      state('closed', style({ height: '0', opacity: 0 })),
      transition('open <=> closed', [
        animate('200ms ease-in-out')
      ]),
    ]),
  ],
  template: `
    <div [@openClose]="isOpen() ? 'open' : 'closed'">
      İçerik
    </div>
  `,
})
export class AccordionComponent {
  isOpen = signal(false);
}
```

□ Enter/Leave

```
animations: [
  trigger('fadeInOut', [
    transition(':enter', [
      style({ opacity: 0, transform: 'translateY(10px)' }),
      animate('200ms ease', style({ opacity: 1, transform: 'translateY(0)' }))
    ]),
    transition(':leave', [
      animate('200ms ease', style({ opacity: 0 }))
    ]),
  ])
]
```

Hangisini Kullanmalı?

Konu	CSS	Angular Animations
Performance	GPU optimized	Daha ağır
Bundle size	0 KB	~20 KB
Complexity	Basit	Karmaşık state
SSR	Sorunsuz	Çalışır
Reactive	CSS only	Trigger ile

Öneri

Çoğu durumda CSS yeterli. @angular/animations sadece complex state-based animations için. Yeni projede CSS-first yaklaşımı tercih et.

Accessibility — Reduce Motion

Kullanıcı vestibular bozuklukları için animasyonları azaltabilir.

🔊 prefers-reduced-motion

```
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    animation-duration: 0.01ms !important;
    animation-iteration-count: 1 !important;
    transition-duration: 0.01ms !important;
  }
}

/* Daha selektif */
.fade-in {
  animation: fadeIn 300ms ease;
}

@media (prefers-reduced-motion: reduce) {
  .fade-in {
    animation: none; /* veya kısa */
  }
}
```

📦 TypeScript'te Detect

```
@Injectable({ providedIn: 'root' })
export class MotionService {
  private mq = window.matchMedia('(prefers-reduced-motion: reduce)');

  prefersReducedMotion = signal(this.mq.matches);

  constructor() {
    this.mq.addEventListener('change', e => {
      this.prefersReducedMotion.set(e.matches);
    });
  }
}

// Component
animationDuration = computed(() =>
  this.motion.prefersReducedMotion() ? 0 : 300
);
```

Performance Best Practices

60fps animasyonlar için altın kurallar.

GPU-Friendly Properties

Property	GPU?	Use
transform	✓	Position, scale, rotate
opacity	✓	Fade
filter	✓	Blur, grayscale
top/left	✗	Layout (kaçın)
width/height	✗	Layout (kaçın)
margin/padding	✗	Layout (kaçın)

will-change Hint

```
/* Browser'a "bu element animate olacak" diyorsun */
.modal {
  will-change: transform, opacity;
  transition: all 300ms ease;
}

/* ⚠ Sadece animation öncesi eklen, sonra kaldır */
.modal.animating {
  will-change: transform;
}

/* Asla sürekli will-change ekleme – memory kullanır */
```

Compose, Don't Animate

```
/* ❌ Kötü – layout */
.element {
  width: 100px;
  animation: grow 300ms ease;
}
@keyframes grow {
  to { width: 200px; }
}

/* ✔ İyi – composite */
.element {
  width: 100px;
  animation: grow 300ms ease;
}
@keyframes grow {
  to { transform: scaleX(2); }
}
```

❏ Devtools — Performance Monitoring

- ✔ Chrome DevTools → Performance ile profiling
- ✔ Rendering tab → Paint flashing, Layout shift regions
- ✔ FPS meter aç — 60fps altına düşmemeli
- ✔ Layers panel → Composite layer'ları gör

Modül 15 Özeti

Animasyon konusunda modern Angular yaklaşımını öğrendin. Çoğu durumda CSS yeterli, complex state için Angular Animations, navigation için View Transitions API.

Neler Öğrendin?

- ✓ CSS animations + transitions (en yaygın yaklaşım)
- ✓ @if / @for ile enter/leave animasyon
- ✓ View Transitions API ile route animations
- ✓ @angular/animations klasik API
- ✓ prefers-reduced-motion accessibility
- ✓ GPU-friendly properties (transform, opacity)
- ✓ will-change ve performance tuning

Sıradaki Modül

Modül 16 — Performance Optimization. Change detection, OnPush, zoneless, bundle analysis, ve daha fazlası.